SIMATS ENGINEERING

Descriptive Written Exam

Course Name: Software Testing **Course Code: CSA37**

Course Outcome Covered: CO1

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Q1. Redesigning Ad-hoc Testing of a Banking Application

Testing is not a random activity performed at the end of development. It is an engineering activity that is planned, designed, executed, measured and improved, just like any other engineering process. In the banking application, frequent failures occur because test cases are unplanned, undocumented and unrepeatable. The solution is to replace ad-hoc testing with a systematic, standards-based approach.

Redesigned Testing Approach

- **Define test objectives and policy:** state what quality means for the bank (accuracy of balances, security, availability).
- **Prepare a test plan:** scope, features to be tested, risks, resources, schedule, entry and exit criteria (IEEE 829 format).
- **Design test cases systematically:** derive them from requirements using proper techniques, with traceability to each requirement.
- **Execute and record:** run tests in a controlled environment, log results and defects in a tracking tool.
- **Measure and improve:** collect metrics such as defect density, test coverage and defect removal efficiency, and use them to improve the next cycle.

Introducing Tools and Standards

Tools remove manual effort and human error: Selenium or JUnit for automated functional and regression tests, JMeter for performance, TestRail for test management, and JIRA or Bugzilla for defect tracking. Standards bring uniformity: IEEE 829 for test documentation, ISO/IEC/IEEE 29119 for the test process, and ISTQB terminology so that everyone uses the same vocabulary.

Techniques to Replace Ad-hoc Testing

Equivalence class partitioning, boundary value analysis, decision tables, state transition testing and use-case testing are applied for black-box design. Statement and branch coverage are applied for white-box design. Error guessing is retained, but only as a supplement to the structured techniques.

Improvement in Defect Detection and Quality

Every requirement is covered by at least one test, tests are repeatable, defects are found earlier when they are cheaper to fix, and coverage can be measured. Quality becomes predictable instead of accidental.

Practical Example: Fund Transfer

Rule: a transfer must be between Rs. 1 and Rs. 50,000 and must not exceed the available balance. An ad-hoc tester may try only Rs. 5,000. A systematic tester uses equivalence classes (below 1 = invalid, 1 to 50,000 = valid, above 50,000 = invalid) and boundary values (0, 1, 2, 49,999, 50,000, 50,001). A decision table combines balance (sufficient or not), limit (within or exceeded) and account status (active or blocked). This design exposes defects such as Rs. 50,000 being wrongly rejected, which random testing would probably miss.

Conclusion

Treating testing as an engineering activity, supported by tools, standards and proven techniques, increases defect detection, reduces production failures and builds customer trust in the banking application.

Q2. Structured Testing Process for an ATM System across the SDLC

Introduction

Inconsistent ATM results show that testing is done without a defined process. A structured process links a testing activity to every SDLC phase (the V-model idea), so that quality is built in and verified at each stage rather than checked only at the end.

Stages of the Process and Their Relevance

SDLC Phase	Testing Activity	Relevance to Quality
Requirements	Review requirements for clarity and testability; prepare acceptance test plan	Removes ambiguity and incomplete requirements early
Design	Review design; prepare system and integration test plans and test cases	Detects design flaws before coding
Coding	Unit testing, code reviews, static analysis	Finds logic and coding defects at the lowest cost
Integration	Integration testing of card reader, PIN module, bank server, cash dispenser	Detects interface defects between modules
System	Functional, performance, security and recovery testing	Verifies the complete ATM against the specification
Acceptance	User and bank acceptance testing	Confirms the system meets business needs
Maintenance	Regression testing after every change	Ensures fixes do not break existing features

Application to ATM Withdrawal Functionality

- **Requirement review:** confirm rules such as three PIN attempts, daily limit, multiples of Rs. 100, minimum balance and cash availability.
- **Unit tests:** PIN validation, balance check and denomination calculation modules.
- **Integration tests:** ATM to bank server messages, debit confirmation, and dispenser command.
- **System tests:** valid withdrawal; wrong PIN three times (card retained); amount above balance; amount above daily limit; amount not a multiple of 100; network failure after debit (amount must be reversed); dispenser out of cash.
- **Regression:** re-run the above whenever the withdrawal logic changes.

How Process Consistency Reduces Defects

A defined process means every release is tested in the same way, with the same entry and exit criteria, templates and defect reporting. Results become repeatable and comparable, nothing

depends on the memory of an individual tester, and gaps are visible. Defects are caught in the phase where they are introduced, so fewer reach production.

Expected Outcomes

Consistent and reliable ATM transactions, fewer production failures, lower rework cost, complete test documentation, measurable quality metrics and higher customer confidence

Q3. Errors, Defects and Failures in a Web Application

Basic Definitions

- **Error (mistake):** a human action that produces an incorrect result, for example a developer misunderstanding a requirement.
- **Defect (fault, bug):** the incorrect code, data or document that results from the error.
- **Failure:** the deviation of the system from its expected behaviour when the defective code is executed.
- **Expected vs. actual result:** the mismatch between the two observed during testing is the symptom of a failure.

Relationship

The chain is: Error leads to Defect, and Defect, when executed, may lead to Failure. Not every defect causes a failure, because the faulty code may never be executed under the tested inputs.

Application to the Scenario

Operation	Login	Transaction
Error	Developer misreads the rule that an account locks after 3 failed attempts	Developer ignores decimal values while designing the amount field
Defect	Code uses 'attempts > 5' instead of 'attempts >= 3'	Amount stored as an integer, so 100.75 becomes 100
Failure	User is not locked after the third wrong password; expected result differs from actual	Account is debited 100 instead of 100.75

Effect of Improper Understanding of the Terms

If testers and developers use these terms loosely, defect reports become vague (for example 'login not working'), the root cause is not traced, the same error type is repeated, and process improvement is impossible because errors, defects and failures cannot be counted separately. Developers may also dispute a report because they cannot tell what was expected and what happened.

Improving Defect Reporting Clarity

- Use a standard template: ID, title, module, environment, steps to reproduce, test data, expected result, actual result, severity, priority, screenshots or logs, and status.
- Report one defect per report, in objective language.
- Example of a poor report: 'Login fails.' Example of a good report: 'Login page accepts a fourth wrong password for user U101. Expected: account locked after the third attempt. Actual: account remains active. Severity: High.'

Justification

Precise terminology and structured reports reduce misunderstanding and re-work, speed up debugging, and allow defects to be classified and analysed, which improves the overall testing process.

Q4. Applying Software Testing Principles to Defect Clustering

Introduction

When most defects are concentrated in a few modules, the team should apply the principles of testing, especially defect clustering, which states that a small number of modules usually contains most of the defects (the Pareto rule: roughly 80% of defects in 20% of modules).

Principles that Guide Test Planning

- **Testing shows the presence of defects:** testing cannot prove the absence of defects, so the focus is on finding them.
- **Exhaustive testing is impossible:** effort must be allocated by risk and priority.
- **Early testing:** reviews and tests start early to prevent defects spreading.
- **Defect clustering:** concentrate additional testing on the defect-prone modules.

- **Pesticide paradox:** repeating the same tests stops finding new defects, so test cases must be reviewed and updated.
- **Testing is context dependent:** a payment module needs more rigorous testing than a help page.
- **Absence-of-errors fallacy:** a defect-free system is still useless if it does not meet user needs.

Prioritising Test Cases

Test cases are ranked using risk (probability of failure multiplied by impact), defect history, complexity of the module, frequency of change, and business criticality. High-priority cases are executed first, and low-risk cases are executed if time permits.

Real-time Example

In an online shopping application, defect data shows that Payment (40%) and Cart (30%) contain 70% of all defects, while Search, Profile and Help share the remaining 30%. The test manager assigns more testers, deeper techniques (boundary value, decision table, integration and negative tests) and extra code review to Payment and Cart, with lighter testing for the others. Priority 1 cases are the checkout, payment failure and coupon calculation scenarios.

Improvement in Testing Efficiency

Effort is directed to where defects are most likely, so more defects are found in less time, delays are reduced, resources are used better and the release is more stable. The team also reviews why those modules are weak (complex logic, frequent changes, inexperienced developers) and corrects the cause.

Q5. Defect Classification and Repository to Prevent Recurring Defects

How Defects Originate

Defects are introduced in every phase of development: unclear or incomplete requirements, wrong design logic, coding mistakes, and also mistakes in test plans and test cases. Causes include poor communication, time pressure, lack of skills, ambiguous documents and changing requirements.

Defect Categories

Class	Description	Example
Requirements / specification	Missing, incorrect or ambiguous functional description or interface	Interest calculation rule not specified
Design	Errors in algorithm, control logic, data structure or module interface	Wrong data structure for account history
Coding	Logic, algorithmic, data, interface or documentation defects	Loop runs one time less
Testing	Faulty test plan, test design or test procedure	Test case missing boundary value

Defects are also categorised by severity (critical, major, minor), priority, module and detection phase.

Using the Defect Repository

The repository stores for each defect: ID, description, class, severity, module, phase of origin, phase of detection, root cause, fix and status. Its data supports the following.

- **Test design:** modules and defect types that occur often receive more test cases; checklists and regression suites are built from past defects.
- **Prevention:** root cause analysis and Pareto charts show the main causes, so reviews, training and coding standards can be targeted.
- **Planning:** historical defect density helps to estimate effort and risk for the next release.

Suggested Improvements

- Make defect entry mandatory and complete, with a standard classification scheme.
- Hold regular defect-analysis meetings and root cause analysis after each release.
- Add a regression test for every fixed defect.
- Update review checklists and coding guidelines using repository findings.
- Monitor trends such as repeat defect rate and defect leakage.

Expected Quality Improvements

Fewer repeated defects, reduced defect leakage into production, better-focused testing, lower maintenance cost and a continuous improvement in software quality across releases.

Q6. Fund Transfer Crashes in a Mobile Banking Application

Introduction

Frequent crashes during fund transfer indicate defects that were not detected before release. Applying testing principles means investigating the failure methodically instead of guessing.

Identifying the Root Cause

- **Reproduce the failure:** record device, OS version, app version, network type, account data and exact steps.
- **Collect evidence:** crash logs, server logs and crash analytics (for example Firebase Crashlytics).
- **Isolate the cause** by changing one factor at a time: device, network, amount, account type.
- **Check likely causes:** unhandled exceptions or null values, memory leaks, timeouts when the network switches from Wi-Fi to mobile data, double tap on the Submit button causing duplicate requests, session expiry, boundary amounts, and incorrect API responses.
- **Apply the 5-Why technique** to reach the underlying cause, for example a missing null check caused by an unvalidated server response.

How Systematic Testing Reduces Such Issues

Planned testing at each level (unit, integration, system) with a coverage target ensures every path of the transfer function is exercised. Using principles such as early testing, risk-based prioritisation and defect clustering, the transfer module is tested more heavily because its failure has the highest impact.

Illustrative Examples

- Transfer of Rs. 0, Rs. 1 and the maximum limit (boundary value analysis).
- Beneficiary with special characters or very long name (invalid input).
- Network interruption after debit and before credit, where the transaction must be rolled back.
- Two rapid taps on Confirm, which must not create two transfers.
- Running the app on low memory and under an incoming call.

Suggested Improvements in Testing Practice

- Maintain a device and OS matrix based on user analytics and use cloud device farms.
- Add network simulation, stress and monkey testing.
- Automate regression tests and run them in a CI pipeline.
- Add exploratory testing sessions and beta testing before release.
- Review production crash data after every release and add tests for each cause.

Q7. Testing versus Debugging

Introduction

Testing and debugging are related but different activities. Confusing them leads to defects being reported without analysis, or being fixed without verification.

Difference between Testing and Debugging

Aspect	Testing	Debugging
Purpose	To detect failures and show the presence of defects	To locate the cause of a defect and remove it
Performed by	Testers (and developers at unit level)	Developers
Input / output	Test cases produce pass or fail results and defect reports	A defect report produces corrected code
Nature	Planned, can be automated, can be done without code knowledge	Reactive, requires detailed code knowledge
Result	Information about quality	A fix, which needs re-testing

Roles in Software Quality

Testing measures quality by revealing defects and giving evidence of risk. Debugging improves quality by correcting them. Quality is achieved only when the cycle test, debug, retest is completed.

Effect of Improper Usage

- Developers who only 'test while debugging' check only the failure they already know about and miss other defects.
- Testers who try to debug waste time and may hide the original failure details.

- Fixes are not re-tested, so defects reappear, and regression defects go unnoticed.
- Effort and schedule estimates are wrong because testing time and debugging time are mixed together.

Structured Approach to Integrate Both

- Tester executes planned tests and logs a clear defect report.
- Defect is triaged and assigned according to severity and priority.
- Developer reproduces, locates the cause using debugging tools and fixes it.
- Developer performs unit testing of the fix.
- Tester performs confirmation testing and regression testing.
- Defect is closed, and the root cause is recorded in the repository for prevention.

Q8. Cross-browser Testing Strategy for an E-commerce Platform

Introduction

Inconsistent behaviour across browsers is a compatibility problem. The strategy applies the principles of risk-based testing, early testing and exhaustive testing being impossible, so only the most relevant browser combinations are selected.

How Defects Originate

- Different rendering engines (Blink, Gecko, WebKit) interpret HTML, CSS and JavaScript differently.
- Use of features or vendor prefixes not supported in all versions.
- Non-standard code and lack of validation against W3C standards.
- Differences in default settings, such as cookies, pop-up blockers, fonts and security policies.
- Different screen sizes, resolutions and operating systems.

Cross-browser Test Strategy

- **Build a compatibility matrix** of browser, version, OS and device using analytics of real users (for example Chrome, Edge, Firefox, Safari on Windows, macOS, Android, iOS).
- **Prioritise** critical flows: search, product page, cart, checkout, payment and login.

- **Define test types:** functional, UI and layout, responsive design, performance and accessibility.
- **Reduce combinations** using pairwise (orthogonal) testing.
- **Execute** first on the most-used browsers, then on the others, and log defects with browser details.

Tools and Techniques for Coverage

- BrowserStack, Sauce Labs and LambdaTest for real browsers and devices.
- Selenium WebDriver and Selenium Grid, Cypress or Playwright for automated tests on multiple browsers.
- Browser developer tools and W3C validators for static checks.
- Visual regression tools to compare screenshots across browsers.
- Techniques: checklists, pairwise testing, progressive enhancement and graceful degradation.

Improvement in Reliability

A planned strategy gives measurable coverage of supported environments, finds compatibility defects before release, ensures a consistent customer experience, and reduces browser-specific complaints and lost sales.

Q9. Classification of Logical Defects in a Payroll System

Introduction

Incorrect salary calculations caused by logical errors in code are coding defects. Classification helps the team to understand what kind of defect has occurred and what testing is needed to find and prevent it.

Categorising the Payroll Defects

Defect Class	Payroll Example	Detection Technique
Control / logic defect	Tax slab condition written as 'salary > 50000' instead of '>= 50000'	Boundary value analysis, branch coverage
Algorithmic defect	HRA calculated on gross salary instead of basic salary	Reviews, unit tests with known results

Defect Class	Payroll Example	Detection Technique
Data defect	Overtime hours not initialised, or amount stored as integer so decimals are lost	Data flow testing, static analysis
Interface defect	Parameters passed in the wrong order to the deduction module	Integration testing
Requirement defect	Leave-without-pay rule misunderstood	Requirement review, acceptance testing

How Understanding Defect Classes Improves Testing

Knowing the classes allows testers to choose suitable techniques (for example boundary tests for logic defects and data-flow tests for data defects), build focused checklists, estimate where defects are likely, and analyse the repository to see which class occurs most often.

Preventing Such Defects in Future Releases

- Peer code reviews and walkthroughs using checklists for conditions and formulas.
- Static analysis tools and coding standards.
- Unit tests for every formula with expected values calculated manually.
- Clear, approved specification of every salary rule and slab.
- Root cause analysis of each payroll defect and training based on it.

Improvements in Testing Strategy

- Use decision tables for combinations of allowances, deductions and tax slabs.
- Apply boundary value analysis at every slab limit.
- Use an independent expected result (manual or spreadsheet calculation) as the test oracle.
- Run parallel testing with the old system's output for a sample of employees.
- Maintain an automated regression suite executed on every change to payroll rules.

Q10. Replacing Random Testing with Structured Test Case Design

Introduction

Executing random tests without design is an unplanned activity, so coverage is unknown and results cannot be repeated. Applying testing as an engineering activity means that test cases are designed, reviewed, documented and traced to requirements before execution.

Redesigned Testing Process

- Analyse requirements and identify test conditions.
- Design test cases using suitable techniques and prepare test data.
- Review the test cases for completeness and correctness.
- Execute tests and compare actual with expected results.
- Log defects, measure coverage and improve the test set.

Importance of Structured Test Case Design

- Gives systematic coverage of requirements, with a traceability matrix.
- Makes tests repeatable, reusable and suitable for automation.
- Avoids duplication and missing cases.
- Allows early review, and allows new testers to execute them.
- Provides data for measuring progress and quality.

A good test case contains an ID, objective, preconditions, input data, steps, expected result and post-conditions.

Improvement in Defect Detection: Login Example

For a username (6 to 12 characters) and password field, random testing might try only one valid and one invalid login. Structured design gives: valid login; username with 5, 6, 12 and 13 characters (boundary values); blank fields; wrong password; special characters; SQL injection string; and account locking after three failures. Each case has an expected result, so wrong behaviour is identified immediately.

Techniques to Enhance Effectiveness

- Black-box: equivalence partitioning, boundary value analysis, decision tables, state transition testing, use-case testing.
- White-box: statement, branch and path coverage.
- Experience-based: error guessing and exploratory testing, as a supplement.

- Reviews of test cases, and periodic updating to avoid the pesticide paradox.

Q11. Data Inconsistency across Modules in a Healthcare Application

Introduction

Data inconsistency in a healthcare system, for example a patient's details differing between registration, laboratory, pharmacy and billing, is a serious quality problem because it can affect patient safety. Testing principles such as early testing, defect clustering at module interfaces and context-dependent testing are applied to analyse it.

How Defects Originate during Development

- Ambiguous or incomplete requirements, and no common data dictionary.
- Different data types, formats or lengths for the same field in different modules (for example date format DD/MM/YYYY and MM/DD/YYYY).
- Interface and data-mapping errors when modules exchange data.
- Missing validation and missing synchronisation or update propagation.
- Duplicate records and concurrent updates without locking.
- Changes made in one module and not communicated to others.

Systematic Testing to Ensure Data Integrity

- **Requirement and design review** of the data model and data dictionary.
- **Integration and interface testing** between modules using the same patient record.
- **Data flow and CRUD testing:** create, read, update and delete in one module and verify the result in all others.
- **Database testing:** referential integrity, constraints, duplicate checks and data migration verification.
- **Boundary, format and negative tests** for each field.
- **Concurrency testing** and end-to-end scenarios such as admission, lab test, prescription and billing.

Example: a patient's date of birth entered in registration is read by billing in a different format and the age-based discount and the dosage check are wrong. An integration test comparing the same field in all modules finds this immediately.

Improvements in Defect Tracking and Reporting

- Use one central defect tracking tool with a standard template and classification.
- Give high severity to defects that affect patient safety or data privacy.
- Link every defect to the requirement, test case and module, and record the root cause.
- Maintain an audit trail of status changes and review open defects daily.
- Report metrics such as defects per module and defects open by severity.

Q12. Using a Defect Repository to Analyse Repeated Defects

Introduction

Repeated defects in spite of testing show that lessons from earlier defects are not being used. A defect repository is a central, organised store of all defect data. When analysed, it shows why defects occur and where testing is weak.

How Repository Data Supports Test Design

- Defect-prone modules and defect types get more and deeper test cases.
- Each fixed defect becomes a regression test, so it cannot return unnoticed.
- Defect classes are used to create review and test checklists.
- Gaps in earlier test cases (defects found late or in production) are identified and corrected.

Identifying Defect Patterns

Data is analysed by module, defect class, severity, phase of origin, phase of detection and root cause, using Pareto charts, trend charts and defect density. For example:

Module	Defects (Release 3)	Observation
Payment	24	Same boundary-value defect repeated in 3 releases
Order management	11	Mostly interface defects
User profile	5	Low risk
Reports	4	Low risk

The table shows clustering in Payment and a recurring boundary defect, which indicates that boundary tests and reviews are missing in that module.

Suggestions to Prevent Recurring Defects

- Perform root cause analysis for every major defect and record corrective action.
- Update regression suites, test design checklists and code review checklists after each release.
- Track metrics such as repeat defect rate, defect leakage and defect removal efficiency.
- Hold defect-prevention meetings, with training for developers on common causes.
- Ensure each defect record is complete and correctly classified so that the analysis is reliable.

Conclusion

Using the repository for analysis moves the team from only finding defects to preventing them, so releases become more stable.

Q13. Performance Failure of a Web-based Learning System at Peak Usage

Introduction

The system fails at peak usage because performance and scalability were not tested. Only functional behaviour at low load was verified. Principles applied here are that exhaustive testing is impossible, so testing must be based on risk, and that testing is context dependent, since a learning system needs load testing during examinations and live classes.

Gaps in Testing

- No load, stress, spike or endurance tests, and no workload model.
- Test environment smaller than production, and unrealistic test data.
- No test of concurrent logins, quiz submissions and video streaming.
- No non-functional requirements such as 'support 5,000 concurrent users with response under 3 seconds'.
- No monitoring of server, database and network resources.

Impact of Test Planning and Coverage on Performance

A test plan must include non-functional coverage, which means the load profiles, peak scenarios, environment, tools, and pass or fail criteria. Without such coverage, bottlenecks such as slow database queries, small connection pools or memory leaks remain hidden until real users find them.

Better Test Strategies

- **Load testing** with expected peak users, for example 5,000 students submitting a quiz at 10:00 a.m.
- **Stress testing** beyond capacity to see how the system fails and recovers.
- **Spike testing** for a sudden increase, and **endurance (soak) testing** for memory leaks.
- **Scalability testing** to verify auto-scaling and load balancing.
- Tools: Apache JMeter, LoadRunner, Gatling and Locust, with monitoring of response time, throughput, error rate and CPU or memory use.

Enhancements to Avoid Failures

- Define performance requirements and acceptance criteria early.
- Use a production-like environment and realistic data volumes.
- Run performance tests in every release cycle and after architecture changes.
- Tune the system using findings: database indexing, caching, content delivery network, connection pooling and horizontal scaling.
- Prepare alerts and a capacity plan for known peak periods.

Q14. Early Involvement of Testers in the SDLC

Introduction

When testers join only after coding, defects that originate in requirements and design are found late. The role of the tester should extend to every phase of the SDLC, following the principle of early testing.

Role of the Tester across the SDLC

Phase	Role of the Tester
Requirements	Review requirements for ambiguity, completeness and testability; define acceptance criteria
Design	Review design, identify risks, start test planning and test case design
Coding	Prepare test data and environment; support unit testing and code reviews
Testing	Execute tests, log defects, perform retesting and regression testing
Release / maintenance	Assess release readiness, support user acceptance, test fixes and changes

Impact of Early Involvement on Quality

The cost of correcting a defect increases sharply as it moves to later phases. Early involvement finds requirement and design defects when they are cheapest to correct, improves testability, gives more time to prepare good test cases, and reduces rework and schedule delays.

Practical Example

A requirement states 'the system should respond quickly'. A tester in the requirements review points out that it is not measurable, and the team changes it to 'search results within 2 seconds for 1,000 users'. Because of this, the design includes caching, and a performance test is planned from the start. If the tester had been absent, the ambiguity would have been found only in system testing, with expensive redesign.

Changes in Team Collaboration Practices

- Include testers in requirement, design and code review meetings.
- Hold joint planning and 'three amigos' sessions (business analyst, developer, tester).
- Adopt Agile or DevOps practices: testers in daily stand-ups, shift-left testing and continuous integration.
- Write acceptance criteria jointly, for example in behaviour-driven development format.
- Share a common tracking tool, and use pair testing between developers and testers.
- Define 'definition of done' to include testing activities.

Q15. Incomplete Test Coverage and Inconsistent System Output

Introduction

If a system works for some inputs and fails for others, the test set did not cover all required behaviour. Coverage is a measure of how much of the requirements, code or input domain has been exercised by the tests.

Identifying the Gaps

- Prepare a requirements traceability matrix to find requirements with no test case.
- Measure code coverage (statement, branch, condition) using tools such as JaCoCo or Coverage.py.

- Review the input domain for untested classes, boundaries and invalid values.
- Review defect history and risk to find weak areas.

How Proper Coverage Ensures Reliability

High and well-planned coverage means that all important requirements, paths and input classes are exercised, so hidden defects are far less likely to remain. Coverage also gives objective evidence of testing completeness and supports decisions about when to stop testing, although 100% coverage does not guarantee absence of defects.

Improving Test Case Design: Example

Consider a grade function: marks 90 and above = A, 75 to 89 = B, 50 to 74 = C, below 50 = Fail. If tests use only 95 and 80, the branches for C and Fail are never run and boundary errors such as '>' instead of '>=' are not seen. An improved design uses equivalence classes (one value from each grade range, plus invalid values such as -1 and 101) and boundary values (49, 50, 74, 75, 89, 90). This gives full branch coverage and finds boundary defects.

Techniques for Better Validation

- Equivalence partitioning, boundary value analysis, decision tables, cause-effect graphing and state transition testing.
- Pairwise testing for combinations of inputs.
- White-box techniques for statement, branch and path coverage.
- Reviews, inspections and static analysis to find defects without execution.
- Mutation testing to check the quality of the test set, and exploratory testing for unexpected cases.

Q16. Improving Defect Reporting in a Logistics Application

Introduction

Poor reporting delays identification and fixing of defects. A clear and consistent report allows the developer to understand, reproduce and correct a defect without further discussion.

Importance of Clear Definitions

When the terms error, defect and failure are clearly defined, the report states what was observed (failure), the cause (defect) and, later, the origin (error). This makes tracking, classification and analysis accurate, and prevents arguments about what is a defect.

Structured Defect Report and Communication

Field	Content
Defect ID and title	Unique number and one-line summary
Module and version	Shipment tracking, release 2.1
Environment	Browser, OS, database, test data
Steps to reproduce	Numbered, precise steps
Expected / actual result	What should happen and what happened
Severity and priority	Impact on the system and urgency of the fix
Attachments	Screenshots, logs, video
Status and owner	New, assigned, fixed, retest, closed, reopened

Example: 'Shipment status not updated after delivery scan. Steps: 1. Log in as courier. 2. Scan parcel P1001 as Delivered. 3. Open the tracking page. Expected: status Delivered. Actual: status remains Out for delivery. Severity: High, Priority: High.' The developer can reproduce this immediately.

Improvements in Reporting Standards

- Adopt a standard template and defect life cycle (IEEE 829 incident report and IEEE 1044 classification).
- Define severity and priority levels with examples so all teams apply them in the same way.
- Make key fields mandatory in a tracking tool such as JIRA or Bugzilla.
- One defect per report, written in objective and neutral language.
- Hold daily defect triage meetings, and monitor aging of open defects.
- Train testers in report writing and review the quality of reports.

Q17. Process-based Testing to Achieve Consistency across Modules

Introduction

Inconsistent testing across modules happens when each tester uses a different method, documents and level of detail. A process-based approach defines one organisation-wide test process that every module follows.

The Standardised Test Process

- Test policy and strategy that define objectives and approach.
- Test planning using a common template (scope, risks, resources, schedule, entry and exit criteria).
- Test design using agreed techniques and a standard test case format.
- Execution, defect reporting and defect life cycle in a common tool.
- Test closure with a standard report, metrics and lessons learned.

How Standardised Processes Improve Quality

Standard processes make results repeatable and comparable between modules, reduce dependence on individual testers, make training easier, ensure that no activity such as reviews or regression is skipped, and provide reliable metrics for management decisions. Maturity models such as TMMi or the Testing Maturity Model provide a roadmap for improving the process in levels.

Illustration

In a learning management system with modules for Registration, Courses, Assessment and Payment, every team follows the same process. Each module has a test plan from the same template; test cases for each requirement designed with equivalence partitioning and boundary value analysis; the same entry criterion (build is stable) and exit criterion (no open critical defects, 95% of tests passed); and the same defect report format. When results are consolidated, the quality of all modules can be compared directly.

Methods to Ensure Consistency

- Common templates and checklists for plans, test cases and reports.
- Standard tools for test management, automation and defect tracking.
- Peer reviews of test artefacts and periodic process audits.
- Training and a central test lead or test process group.
- Defined metrics reviewed regularly, with process improvement actions.

Q18. Failures Caused by Untested Edge Cases

Introduction

A system that fails after deployment because of edge cases shows that testing was concentrated on typical inputs. Edge cases are inputs at the extremes or limits of the permitted range, and defects tend to cluster there.

Why Edge Cases Were Missed

- Testing concentrated on the normal (happy) path.
- Requirements did not state limits clearly, so test cases were not designed for them.
- Boundary value analysis and negative testing were not used.
- Time pressure and the belief that exhaustive testing is possible, which it is not, so risk-based selection should have been used.
- Limited test data that did not include extreme values.

How Boundary Testing Helps

Programmers frequently make off-by-one mistakes, such as using '<' instead of '<='. Boundary value analysis tests values at and around each limit, which is where these errors appear. For a valid range min to max, the tests are min-1, min, min+1, max-1, max and max+1.

Example: an insurance application accepts ages 18 to 60. The boundary tests are 17, 18, 19, 59, 60 and 61. Only 6 tests find limit errors that a test of ages such as 30 or 40 would never find.

Including Edge Cases in Testing

- Identify limits in each requirement (range, length, size, count, date, time).
- Add other edge cases: empty and null values, zero, negative numbers, maximum length strings, special characters, very large files, leap year, month end, time zone changes, and low memory or storage.
- Prepare test data for each case, and add it to the traceability matrix.
- Use error guessing and exploratory testing based on past defects.

Improvements in Test Design

Combine equivalence partitioning with boundary value analysis, use robust boundary testing with invalid values, add an edge case checklist to the test design review, automate boundary tests for regression, and update the checklist from every production defect.

Q19. Defect Documentation and the Defect Repository

Introduction

Without proper documentation of defects, issues are lost, duplicated or repeated, and nobody knows their status. A defect repository solves this by storing all defect information in a single, structured and searchable place.

How Documentation Improves Defect Management

- **Tracking:** each defect has an ID, owner and status through its life cycle.
- **Communication:** testers, developers and managers see the same information.
- **Avoiding duplicates:** the team can search before logging a new defect.
- **Accountability and audit:** history of who changed what and when.
- **Knowledge retention:** information is available even when team members change.

Structured Repositories Supporting Analysis

Repository Data	Analysis Supported
Module, class and severity	Identify defect-prone modules and the most common types
Phase of origin and detection	Measure defect removal efficiency and find weak phases
Root cause	Plan prevention actions and training
Open date, fix date, status	Measure aging, fix time and backlog trends
Link to requirement and test case	Check coverage and impact of changes

Reports and charts derived from this data (Pareto chart, trend of open defects, defect density) help managers to take decisions about release and resources.

Improvements in Documentation Practices

- Introduce a standard defect template with mandatory fields and a clear life cycle.
- Use a tool such as JIRA, Bugzilla or Redmine, with access control and change history.
- Attach evidence (screenshots, logs) and link defects to requirements and test cases.
- Define severity, priority and classification guidelines.
- Review the repository regularly, close or merge stale and duplicate records, and train the team.

Q20. Redesigning the Testing Approach for a Banking Transaction System

Introduction

Repeated failures in transaction processing show that testing is not designed according to the nature of the system. Banking transactions must satisfy the ACID properties: Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent transactions do not interfere) and Durability (committed data is permanent). The testing approach is redesigned around them using the principles of testing.

Redesigned Structured Approach

- Risk analysis to identify critical transactions (transfer, deposit, withdrawal, bill payment).
- A test plan with objectives, test levels, environment, schedule and exit criteria.
- Test design using defined techniques and traceability to requirements.
- Automated execution with data-driven tests, and a defect tracking and regression process.
- Metrics and root cause analysis after each cycle.

How Structured Testing Improves Quality

Each requirement and risk is covered by planned tests, tests are repeatable through automation, defects are found at unit and integration levels before system testing, and the results provide objective information on release readiness.

Illustrative Examples

- **Atomicity:** transfer Rs. 5,000 and force a failure after the debit; the debit must be rolled back.
- **Isolation:** two users withdraw from the same account at the same time; the balance must stay correct.
- **Boundary:** transfer exactly equal to the daily limit, and limit plus Rs. 1.
- **Duplicate request:** repeated submission after a timeout must not create two transactions.
- **Durability:** after a server restart, committed transactions are still present.

Techniques to Enhance Defect Detection •

Equivalence partitioning, boundary value analysis, decision tables and state transition testing (pending, success, failed, reversed).

- Negative, security and concurrency testing, and performance testing under peak load.
- Reconciliation tests comparing totals of debits and credits.
- Code reviews, static analysis and code coverage measurement.
- Regression automation in a continuous integration pipeline and repository-driven test improvement.

Conclusion

A risk-based, technique-driven and measured approach improves detection of defects in transaction processing and gives reliable, accurate and secure banking services.

Code of Conduct:

I K.Sai vardhan **(**192311017**)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Sai vardhan

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand